

Digital Logic I

Michael C. Hackett

Associate Professor, Computer Science



Lecture Topics

- Boolean Functions
- Boolean Expressions
- Simplifying Boolean Expressions
 - Karnaugh Maps
- Logic Gates and Circuits
- Buses
- NAND/NOR-Only Gates

Boolean Functions

- We've become familiar with logical operations (and, or, not...) while working with and assembly language (and high-level languages in other courses)
- With more complex logical expressions, we use special notations to represent Boolean functions.
 - Some of these notations were seen in a previous lecture

Boolean Functions

$$x \text{ AND } y$$

may be expressed as either:

$$x \cdot y$$
$$xy$$

x	y	$x \text{ AND } y$
0	0	0
0	1	0
1	0	0
1	1	1

Boolean Functions

$x \text{ OR } y$
is expressed as:

$$x + y$$

x	y	$x \text{ OR } y$
0	0	0
0	1	1
1	0	1
1	1	1

Boolean Functions

NOT x

may be expressed as either:

x' ("*x prime*")

$\overline{x}$ ("*x bar*" or "*not x*")

x	<i>NOT x</i>
0	1
1	0

Boolean Functions

$x \text{ XOR } y$
is expressed as:

$$\bar{x}y + x\bar{y}$$
$$x \oplus y$$

x	y	$x \text{ XOR } y$
0	0	0
0	1	1
1	0	1
1	1	0

Boolean Functions

- An **NOR** operation results in true (1) only when **both** of the operands are false (0)
 - It is the negation of the **or** operator

$x \text{ NOR } y$

is expressed as:

$$\overline{x + y}$$

x	y	$x \text{ NOR } y$
0	0	1
0	1	0
1	0	0
1	1	0

Boolean Functions

- A **NAND** operation results in true (1) only when **both** of the operands are *not* true (1)
 - It is the negation of the **and** operator

$x \text{ NAND } y$

may be expressed as either:

$$\overline{x \cdot y}$$

$$\overline{xy}$$

x	y	$x \text{ NAND } y$
0	0	1
0	1	1
1	0	1
1	1	0

Boolean Functions

- An **XNOR** (exclusive **nor**, “*x nor*”; “*znor*”) operation results in true (1) only when the operands are either both true (1) or both false (0)
 - It is the negation of the **xor** operator

$x \text{ XNOR } y$

is expressed as:

$$\bar{x} \bar{y} + xy$$

$$x \odot y$$

x	y	$x \text{ XNOR } y$
0	0	1
0	1	0
1	0	0
1	1	1

Boolean Functions

- The precedence for Boolean operators is
 1. *Expressions in parentheses*
 2. *NOT*
 3. *AND*
 4. *OR*
- $x \text{ NOR } y = \text{NOT}(x \text{ OR } y)$
- $x \text{ NAND } y = \text{NOT}(x \text{ AND } y)$
- $x \text{ XNOR } y = \text{NOT}(x \text{ XOR } y)$
- $x \text{ XOR } y = (\text{NOT}(x) \text{ AND } y) \text{ OR } (x \text{ AND } \text{NOT}(y))$

Boolean Expressions

- Boolean expressions can be evaluated using truth tables
 - Be sure to follow the order of operations

$$\overline{x} + y$$

		1	2
x	y	$\overline{x}$	$\overline{x} + y$
0	0	1	1
0	1	1	1
1	0	0	0
1	1	0	1

Boolean Expressions

- Boolean expressions can be evaluated using truth tables

$$x + xy$$

		1	2
x	y	xy	$x + xy$
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

Boolean Expressions

- Boolean expressions can be evaluated using truth tables

$$\bar{x} + xy + \bar{y}$$

		1	2	3	4	5
x	y	$\bar{x}$	$\bar{y}$	xy	$\bar{x} + xy$	$\bar{x} + xy + \bar{y}$
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	0	1
1	1	0	0	1	1	1

- This truth table tells us that, regardless of the value of x or y, the result will always be 1

Boolean Expressions

- Boolean expressions can be evaluated using truth tables

$$x + y\bar{z}$$

			1	2	3
x	y	z	$\bar{z}$	$y\bar{z}$	$x + y\bar{z}$
0	0	0	1	0	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	0	0	0
1	0	0	1	0	1
1	0	1	0	0	1
1	1	0	1	1	1
1	1	1	0	0	1

Boolean Expressions

- We'll take the previous expressions and convert them to **sum-of-products form** (or, *disjunctive normal form*)
- This form can be obtained with a truth table
- We are interested in the scenarios where the output of the function is 1 (true)

Boolean Expressions

- Converting to Sum-of-Products

$$\bar{x} + y$$

x	y	$\bar{x}$	$\bar{x} + y$
0	0	1	1
0	1	1	1
1	0	0	0
1	1	0	1

$$\bar{x}\bar{y} + \bar{x}y + xy$$

Boolean Expressions

- Proving their identity

$$\bar{x} + y = \bar{x}\bar{y} + \bar{x}y + xy$$

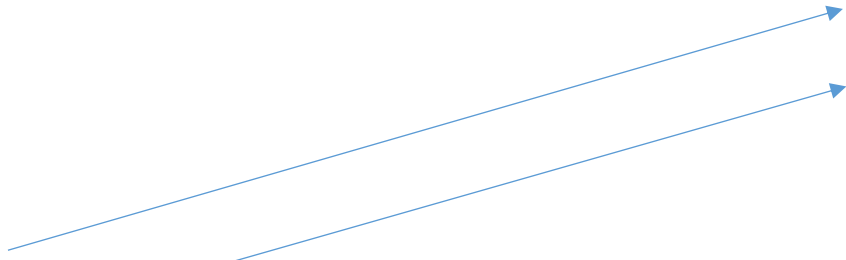
Original					SOP Form			
x	y	$\bar{x}$	$\bar{y}$	$\bar{x} + y$	$\bar{x}\bar{y}$	$\bar{x}y$	xy	$\bar{x}\bar{y} + \bar{x}y + xy$
0	0	1	1	1	1	0	0	1
0	1	1	0	1	0	1	0	1
1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	1	1

Boolean Expressions

- Converting to Sum-of-Products

$$x + xy$$

x	y	xy	$x + xy$
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

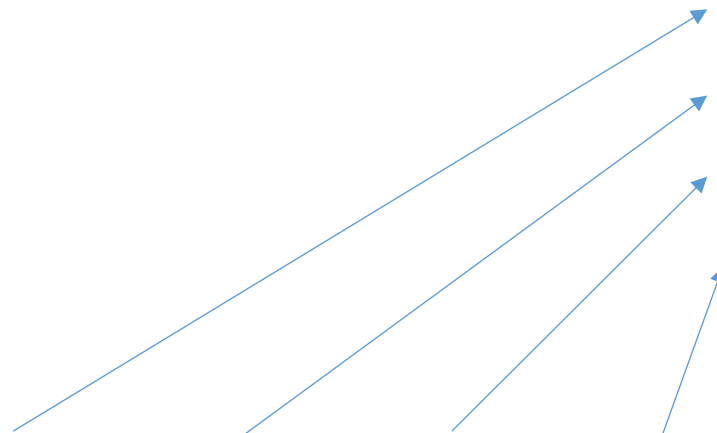
$$x\bar{y} + xy$$


Boolean Expressions

- Converting to Sum-of-Products

$$\bar{x} + xy + \bar{y}$$

x	y	$\bar{x}$	$\bar{y}$	xy	$\bar{x} + xy$	$\bar{x} + xy + \bar{y}$
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	0	1
1	1	0	0	1	1	1

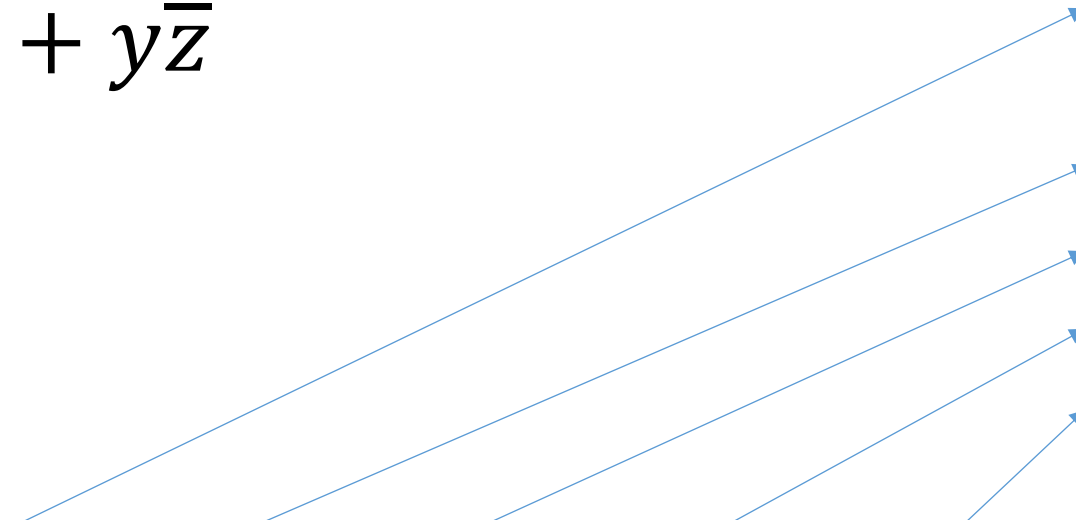
$$\bar{x}\bar{y} + \bar{x}y + x\bar{y} + xy$$


Boolean Expressions

- Converting to Sum-of-Products

$$x + y\bar{z}$$

x	y	z	$\bar{z}$	$y\bar{z}$	$x + y\bar{z}$
0	0	0	1	0	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	0	0	0
1	0	0	1	0	1
1	0	1	0	0	1
1	1	0	1	1	1
1	1	1	0	0	1

$$\bar{x}y\bar{z} + x\bar{y}\bar{z} + x\bar{y}z + xy\bar{z} + xyz$$


Boolean Expressions

- We are now ready to look at some laws of Boolean algebra that will let us simplify Boolean expressions to the fewest terms and operations needed.

$$\bar{x} + y = \bar{x}\bar{y} + \bar{x}y + xy$$

- Though these expressions are logically equivalent, the first expression uses fewer terms and operations than the second.

Boolean Expressions

- Identity Laws

$$x \cdot 1 = x$$

$$x + 0 = x$$

x	$x \cdot 1$	$x + 0$
0	0	0
1	1	1

Boolean Expressions

- Constant Laws

$$x \cdot 0 = 0$$

$$x + 1 = 1$$

x	$x \cdot 0$	$x + 1$
0	0	1
1	0	1

Boolean Expressions

- Negation Laws

$$x \cdot \bar{x} = 0$$

$$x + \bar{x} = 1$$

x	$\bar{x}$	$x \cdot \bar{x}$	$x + \bar{x}$
0	1	0	1
1	0	0	1

Boolean Expressions

- Double Negation Law

$$\overline{\overline{x}} = x$$

x	$\overline{x}$	$\overline{\overline{x}}$
0	1	0
1	0	1

Boolean Expressions

- Idempotent Laws

$$x \cdot x = x$$

$$x + x = x$$

x	$x \cdot x$	$x + x$
0	0	0
1	1	1

Boolean Expressions

- Commutative Laws

$$x \cdot y = y \cdot x$$

$$x + y = y + x$$

x	y	$x \cdot y$	$y \cdot x$
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

x	y	$x + y$	$y + x$
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Boolean Expressions

- Distributive Laws

$$x \cdot (y + z) = xy + xz$$

x	y	z	$(y + z)$	$x \cdot (y + z)$
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

x	y	z	xy	xz	$xy + xz$
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	0	0	0
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	1	1	1

Boolean Expressions

- Distributive Laws

$$x + (yz) = (x + y) \cdot (x + z)$$

x	y	z	(yz)	$x + (yz)$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

x	y	z	$(x + y)$	$(x + z)$	$(x + y) \cdot (x + z)$
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	1	1	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	1	1	1
1	1	1	1	1	1

Boolean Expressions

- Associative Laws

$$x \cdot (y \cdot z) = (x \cdot y) \cdot z$$

x	y	z	$(y \cdot z)$	$x \cdot (y \cdot z)$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

x	y	z	$(x \cdot y)$	$(x \cdot y) \cdot z$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	0	0
1	1	0	1	0
1	1	1	1	1

Boolean Expressions

- Associative Laws

$$x + (y + z) = (x + y) + z$$

x	y	z	$(y + z)$	$x + (y + z)$
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	0	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

x	y	z	$(x + y)$	$(x + y) + z$
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

Boolean Expressions

- DeMorgan's Laws

$$\overline{x + y} = \bar{x} \cdot \bar{y}$$

x	y	$x + y$	$\overline{x + y}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

x	y	$\bar{x}$	$\bar{y}$	$\bar{x} \cdot \bar{y}$
0	0	1	1	1
0	1	1	0	0
1	0	0	1	0
1	1	0	0	0

Boolean Expressions

- DeMorgan's Laws

$$\overline{x \cdot y} = \bar{x} + \bar{y}$$

x	y	$x \cdot y$	$\overline{x \cdot y}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

x	y	$\bar{x}$	$\bar{y}$	$\bar{x} + \bar{y}$
0	0	1	1	1
0	1	1	0	1
1	0	0	1	1
1	1	0	0	0

Boolean Expressions

- Absorption Laws

$$x + (x \cdot y) = x$$

$$x \cdot (x + y) = x$$

x	y	$x \cdot y$	$x + (x \cdot y)$
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

x	y	$x + y$	$x \cdot (x + y)$
0	0	0	0
0	1	1	0
1	0	1	1
1	1	1	1

Boolean Expressions

- Absorption Laws

$$x + (\bar{x} \cdot y) = x + y$$

$$x \cdot (\bar{x} + y) = x \cdot y$$

x	y	$\bar{x}$	$x + y$	$\bar{x} \cdot y$	$x + (\bar{x} \cdot y)$
0	0	1	0	0	0
0	1	1	1	1	1
1	0	0	1	0	1
1	1	0	1	0	1

x	y	$\bar{x}$	$x \cdot y$	$\bar{x} + y$	$x \cdot (\bar{x} + y)$
0	0	1	0	1	0
0	1	1	0	1	0
1	0	0	0	0	0
1	1	0	1	1	1

Boolean Expressions

- Consensus Laws

$$xy + \bar{x}z + yz = xy + \bar{x}z$$

x	y	z	$\bar{x}$	xy	$\bar{x}z$	yz	$xy + \bar{x}z$	$xy + \bar{x}z + yz$
0	0	0	1	0	0	0	0	0
0	0	1	1	0	1	0	1	1
0	1	0	1	0	0	0	0	0
0	1	1	1	0	1	1	1	1
1	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0
1	1	0	0	1	0	0	1	1
1	1	1	0	1	0	1	1	1

Boolean Expressions

- Consensus Laws

$$(x + y) \cdot (\bar{x} + z) \cdot (y + z) = (x + y) \cdot (\bar{x} + z)$$

x	y	z	$\bar{x}$	$x + y$	$\bar{x} + z$	$y + z$	$(x + y) \cdot (\bar{x} + z)$	$(x + y) \cdot (\bar{x} + z) \cdot (y + z)$
0	0	0	1	0	1	0	0	0
0	0	1	1	0	1	1	0	0
0	1	0	1	1	1	1	1	1
0	1	1	1	1	1	1	1	1
1	0	0	0	1	0	0	0	0
1	0	1	0	1	1	1	1	1
1	1	0	0	1	0	1	0	0
1	1	1	0	1	1	1	1	1

Boolean Expressions

- An earlier example:

$$\bar{x} + y = \bar{x}\bar{y} + \bar{x}y + xy$$

- Let's use any applicable laws to the SOP expression:

$$\bar{x}\bar{y} + \bar{x}y + xy$$

- To arrive at its equivalent:

$$\bar{x} + y$$

Boolean Expressions

- Distributive Law:
 $x \cdot (y + z) = \mathbf{xy} + \mathbf{xz}$

$$\bar{x}\bar{y} + \bar{x}y + xy$$

$$\bar{\mathbf{x}}\bar{\mathbf{y}} + \bar{\mathbf{x}}\mathbf{y} + \mathbf{xy}$$

- Negation Law:
 $\mathbf{x} + \bar{\mathbf{x}} = 1$

$$\bar{x}(\bar{y} + y) + xy$$

$$\bar{x}(\bar{\mathbf{y}} + \mathbf{y}) + \mathbf{xy}$$

- Identity Law:
 $\mathbf{x} \cdot \mathbf{1} = \mathbf{x}$

$$\bar{x}(1) + xy$$

$$\bar{\mathbf{x}}(\mathbf{1}) + \mathbf{xy}$$

- Absorption Law:
 $\mathbf{x} + (\bar{\mathbf{x}} \cdot \mathbf{y}) = \mathbf{x} + \mathbf{y}$

$$\bar{x} + xy$$

$$\bar{\mathbf{x}} + \mathbf{xy}$$

$$\bar{x} + y$$

Boolean Expressions

- An earlier example:

$$x + xy = x\bar{y} + xy$$

was proven to be true with a truth table

- Let's use any applicable laws to this SOP expression to see if the Boolean expression can be simplified further:

$$x\bar{y} + xy$$

Boolean Expressions

Distributive Law:

$$x \cdot (y + z) = \mathbf{xy} + \mathbf{xz}$$

$$x\bar{y} + xy$$

$$\mathbf{x\bar{y}} + \mathbf{xy}$$

- Negation Law:

$$\mathbf{x} + \bar{\mathbf{x}} = 1$$

$$x(\bar{y} + y)$$

$$x(\bar{\mathbf{y}} + \mathbf{y})$$

- Identity Law:

$$\mathbf{x} \cdot \mathbf{1} = x$$

$$x(1)$$

$$\mathbf{x(1)}$$

$$x$$

Boolean Expressions

$$x + xy = x\bar{y} + xy = x$$

x	y	$\bar{y}$	xy	$x\bar{y}$	$x + xy$	$x\bar{y} + xy$
0	0	1	0	0	0	0
0	1	0	0	0	0	0
1	0	1	0	1	1	1
1	1	0	1	1	1	1

Boolean Expressions

- An earlier example:

$$\bar{x} + xy + \bar{y} = \bar{x}\bar{y} + \bar{x}y + x\bar{y} + xy$$

- We not only proved this to be true with truth tables, but we also saw the result is always 1

x	y	$\bar{x}$	$\bar{y}$	xy	$\bar{x} + xy$	$\bar{x} + xy + \bar{y}$
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	0	1
1	1	0	0	1	1	1

- Let's also prove it true using any applicable laws

Boolean Expressions

- Distributive Law:
 $x \cdot (y + z) = xy + xz$

$$\bar{x}\bar{y} + \bar{x}y + x\bar{y} + xy$$

$$\bar{x}\bar{y} + \bar{x}y + x\bar{y} + xy$$
 - Negation Law:
 $x + \bar{x} = 1$

$$\bar{x}(\bar{y} + y) + x\bar{y} + xy$$

$$\bar{x}(\bar{y} + y) + x\bar{y} + xy$$
 - Identity Law:
 $x \cdot 1 = x$

$$\bar{x}(1) + x\bar{y} + xy$$

$$\bar{x}(1) + x\bar{y} + xy$$
- $$\bar{x} + x\bar{y} + xy$$

- Distributive Law:
 $x \cdot (y + z) = xy + xz$

$$\bar{x} + x\bar{y} + xy$$

$$\bar{x} + x\bar{y} + xy$$
 - Negation Law:
 $x + \bar{x} = 1$

$$\bar{x} + x(\bar{y} + y)$$

$$\bar{x} + x(\bar{y} + y)$$
 - Identity Law:
 $x \cdot 1 = x$

$$\bar{x} + x(1)$$

$$\bar{x} + x(1)$$
 - Negation Law:
 $x + \bar{x} = 1$

$$\bar{x} + x$$

$$\bar{x} + x$$
- $$1$$

Boolean Expressions

- An earlier example:

$$x + y\bar{z} = \bar{x}y\bar{z} + x\bar{y}\bar{z} + x\bar{y}z + xy\bar{z} + xyz$$

was proven to be true with a truth table

- Let's use any applicable laws to the SOP expression:

$$\bar{x}y\bar{z} + x\bar{y}\bar{z} + x\bar{y}z + xy\bar{z} + xyz$$

- To arrive at its equivalent:

$$x + y\bar{z}$$

Boolean Expressions

- Distributive Law:

$$\bar{x}y\bar{z} + x\bar{y}\bar{z} + x\bar{y}z + xy\bar{z} + xyz$$
$$x(\bar{y}\bar{z} + \bar{y}z + y\bar{z} + yz) + \bar{x}y\bar{z}$$

- Distributive Law (x2):

$$x(\bar{y}\bar{z} + \bar{y}z + y\bar{z} + yz) + \bar{x}y\bar{z}$$
$$x(\bar{y}(\bar{z} + z) + y(\bar{z} + z)) + \bar{x}y\bar{z}$$

- Negation Law (x2):

$$x(\bar{y}(\bar{z} + z) + y(\bar{z} + z)) + \bar{x}y\bar{z}$$
$$x(\bar{y}(1) + y(1)) + \bar{x}y\bar{z}$$

- Identity Law (x2):

$$x(\bar{y} + y) + \bar{x}y\bar{z}$$

- Negation Law:

$$x(\bar{y} + y) + \bar{x}y\bar{z}$$
$$x(1) + \bar{x}y\bar{z}$$

- Identity Law:

$$x + \bar{x}y\bar{z}$$

- Absorption Law:

$$x + (\bar{x} \cdot y) = x + y$$

$$x + \bar{x}y\bar{z}$$

$$x + y\bar{z}$$

Simplifying Boolean Expressions

- Those laws are useful for reducing expressions, but it can be cumbersome to reduce very complex expression this way
- A tool used to help with the simplification process is a Karnaugh Map
 - K-Map, for short
- K-Maps are not a total substitute for Boolean algebra, but they are faster

Simplifying Boolean Expressions

- Below is a two-variable K-Map
 - Notice that only one bit changes between each row and column

		y	
		$\bar{y}$	y
x	$\bar{x}$ 0	00	01
	x 1	10	11

Simplifying Boolean Expressions

- To demonstrate, we'll use the following expression:

$$x + \bar{y}$$

- Which yields the following truth table:

x	y	$\bar{y}$	$x + \bar{y}$
0	0	1	1
0	1	0	0
1	0	1	1
1	1	0	1

Simplifying Boolean Expressions

x	y	$\bar{y}$	$x + \bar{y}$
0	0	1	1
0	1	0	0
1	0	1	1
1	1	0	1

		y	
		$\bar{y}$	y
x	$\bar{x}$	0	1
	0	1 00	01
x	1	10	11

Simplifying Boolean Expressions

x	y	$\bar{y}$	$x + \bar{y}$
0	0	1	1
0	1	0	0
1	0	1	1
1	1	0	1

		y	
		$\bar{y}$	y
x	$\bar{x}$	0	1
	0	1 00	0 01
x	1	10	11

Simplifying Boolean Expressions

x	y	$\bar{y}$	$x + \bar{y}$
0	0	1	1
0	1	0	0
1	0	1	1
1	1	0	1

		y	
		$\bar{y}$ 0	y 1
x	$\bar{x}$ 0	1 00	0 01
	x 1	1 10	 11

Simplifying Boolean Expressions

x	y	$\bar{y}$	$x + \bar{y}$
0	0	1	1
0	1	0	0
1	0	1	1
1	1	0	1

		y	
		$\bar{y}$	y
x	$\bar{x}$	0	1
	0	1 00	0 01
	x	1	1
	1	1 10	1 11

Simplifying Boolean Expressions

- K-Map for $x + \bar{y}$

		<i>y</i>	
		$\bar{y}$	y
<i>x</i>	$\bar{x}$ 0	1 00	0 01
	x 1	1 10	1 11

Simplifying Boolean Expressions

- Resulting SOP Expression: $\bar{x} \bar{y} + x\bar{y} + xy$

		y	
		$\bar{y}$	y
x	$\bar{x}$ 0	1 00	0 01
	x 1	1 10	1 11

Simplifying Boolean Expressions

- When using K-Maps, we can group adjacent cells together in order to simplify the function

		y	
		$\bar{y}$	y
x	$\bar{x}$ 0	1 00	0 01
	x 1	1 10	1 11

- This illustrates the output is 1 when $y = 0$ (it does not depend on x)

Simplifying Boolean Expressions

- When grouping adjacent cells, you must group them in powers of 2
 - A group of 2, 4, 8, etc.
- The grouped cells must all contain 1s
- Cells cannot be grouped diagonally

Simplifying Boolean Expressions

		y	
		$\bar{y}$	y
x	$\bar{x}$ 0	1 00	0 01
	x 1	1 10	1 11

$\bar{y} + xy$

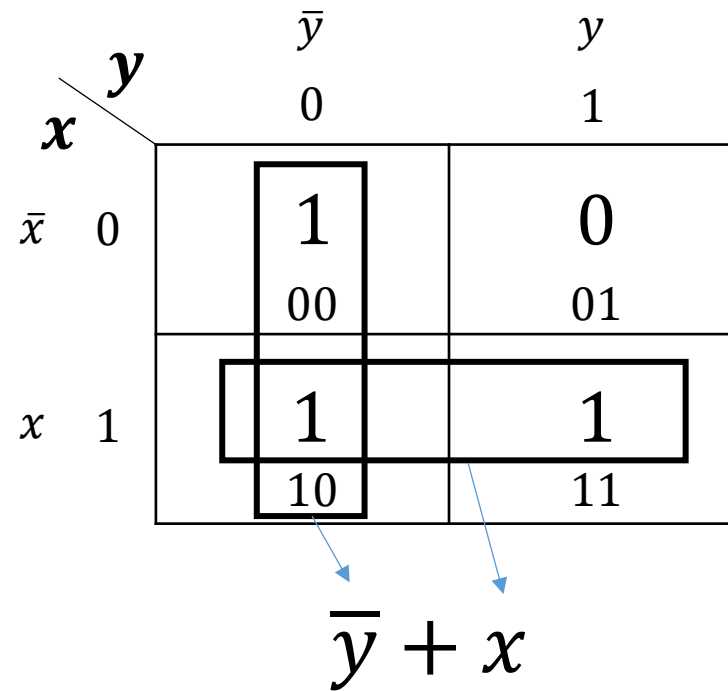
Simplifying Boolean Expressions

- We can reuse 1's that were grouped with other adjacent cells

		y	
		$\bar{y}$	y
x	$\bar{x}$	0	1
	0	1 00	0 01
x	1	1 10	1 11

- This second grouping illustrates the output is 1 when $x = 1$ (it does not depend on y)

Simplifying Boolean Expressions



- Our original expression was $x + \bar{y}$
 - Already reduced to fewest terms

Simplifying Boolean Expressions

- Here's an SOP Expression from earlier: $\bar{x}\bar{y} + \bar{x}y + x\bar{y} + xy$
 - The one that always results in 1

		y	
		$\bar{y}$	y
x	$\bar{x}$ 0	1 00	1 01
	x 1	1 10	1 11

Simplifying Boolean Expressions

- Here, we've grouped all 4 (2^2) cells together
 - Just like we saw earlier, it doesn't matter what the values of x and y are- the function always results in 1

		y	
		$\bar{y}$	y
x	$\bar{x}$ 0	1 00	1 01
	x 1	1 10	1 11

Simplifying Boolean Expressions

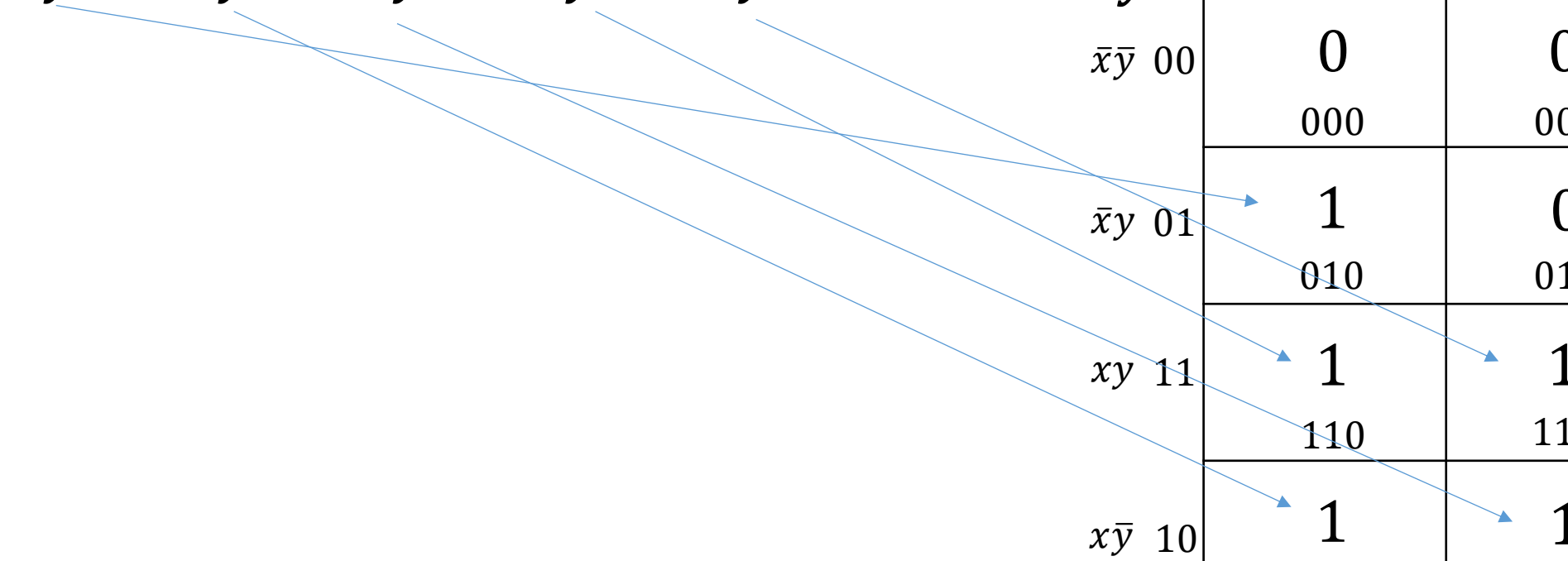
- This is a three-variable K-Map
 - Notice that only one bit changes between each row and column

		z	
		$\bar{z}$ 0	z 1
x y	$\bar{x}\bar{y}$ 00	0 000	0 001
	$\bar{x}y$ 01	0 010	0 011
	xy 11	0 110	0 111
	$x\bar{y}$ 10	0 100	0 101

Simplifying Boolean Expressions

- Here's another SOP Expression from earlier:

$$\bar{x}y\bar{z} + x\bar{y}\bar{z} + x\bar{y}z + xy\bar{z} + xyz$$



A Karnaugh map for three variables x, y, and z. The map is a 4x2 grid. The columns are labeled $\bar{z}$ (0) and z (1). The rows are labeled $x\bar{y}$ (00), $\bar{x}y$ (01), xy (11), and $x\bar{y}$ (10). The cells contain the values 0 or 1, with the 3-bit binary representation of the cell index below the value. Blue arrows point from the terms in the SOP expression to the cells where they are 1: $\bar{x}y\bar{z}$ points to cell 010, $x\bar{y}\bar{z}$ points to cell 100, $x\bar{y}z$ points to cell 101, $xy\bar{z}$ points to cell 110, and xyz points to cell 111.

		$\bar{z}$	z
		0	1
$x\bar{y}$	00	0 000	0 001
	01	1 010	0 011
xy	11	1 110	1 111
	10	1 100	1 101

Simplifying Boolean Expressions

- First grouping (4 cells)
- Result is 1 when x is 1
 - y and z are irrelevant

		z	
		$\bar{z}$ 0	z 1
x y	$\bar{x}\bar{y}$ 00	0 000	0 001
	$\bar{x}y$ 01	1 010	0 011
	xy 11	1 110	1 111
	$x\bar{y}$ 10	1 100	1 101

Simplifying Boolean Expressions

- Second grouping (2 cells)
- Result is 1 when y is 1 and z is 0
 - x is irrelevant

		z	
		$\bar{z}$ 0	z 1
x y	$\bar{x}\bar{y}$ 00	0 000	0 001
	$\bar{x}y$ 01	1 010	0 011
	xy 11	1 110	1 111
	$x\bar{y}$ 10	1 100	1 101

Simplifying Boolean Expressions

- Result:

$$x + y\bar{z}$$

(Same as was derived with Boolean algebra)

$\begin{matrix} & \mathbf{z} \\ \mathbf{x} \ \mathbf{y} & \end{matrix}$		$\bar{z}$	z
		0	1
$\bar{x}\bar{y}$	00	0 000	0 001
$\bar{x}y$	01	1 010	0 011
xy	11	1 110	1 111
$x\bar{y}$	10	1 100	1 101

Simplifying Boolean Expressions

- Another (new) SOP Expression:
 $x\bar{y}z + \bar{x}\bar{y}z$

$\begin{matrix} & z \\ x & y \end{matrix}$		$\bar{z}$	z
		0	1
$\bar{x}\bar{y}$	00	0 000	1 001
$\bar{x}y$	01	0 010	0 011
xy	11	0 110	0 111
$x\bar{y}$	10	0 100	1 101

Simplifying Boolean Expressions

- Cell groups can wrap around the map
- Result is 1 when z is 1 and y is 0
 - x is irrelevant
- Result:

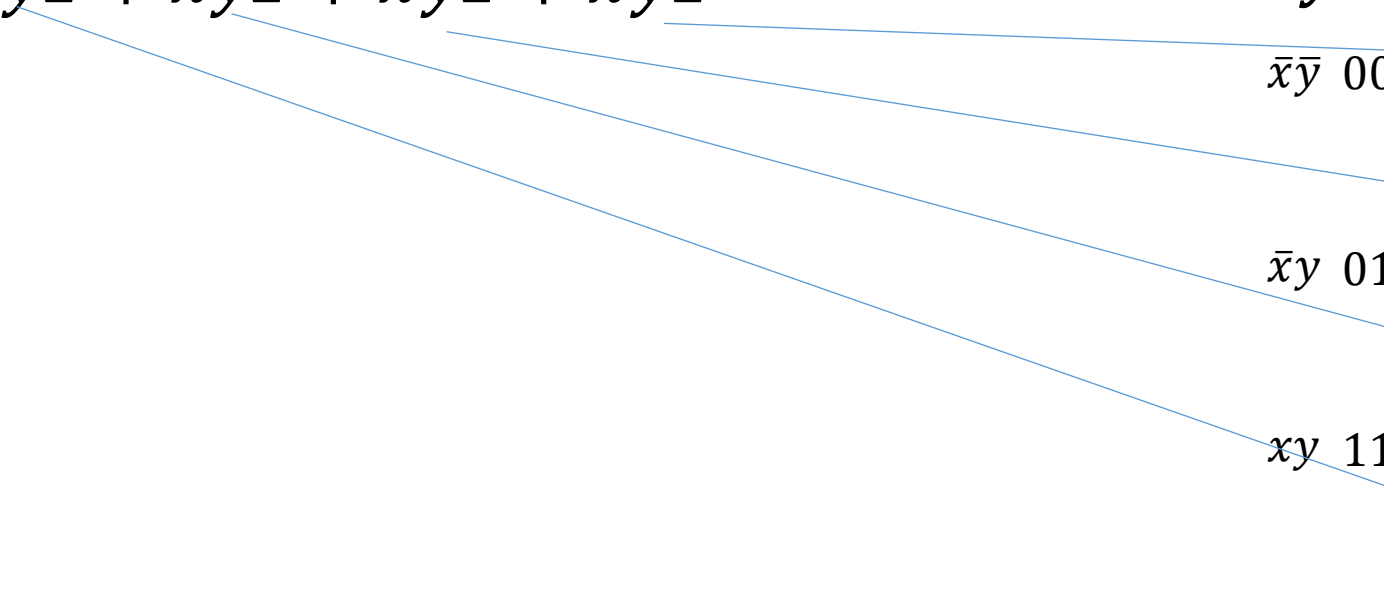
$$x\bar{y}z + \bar{x}\bar{y}z = \bar{y}z$$

$\begin{matrix} & z \\ x & y \end{matrix}$		$\bar{z}$	z
		0	1
$\bar{x}\bar{y}$	00	0 000	1 001
$\bar{x}y$	01	0 010	0 011
xy	11	0 110	0 111
$x\bar{y}$	10	0 100	1 101

Simplifying Boolean Expressions

- Another (new) SOP Expression:

$$x\bar{y}z + xyz + \bar{x}yz + \bar{x}\bar{y}z$$



		z	
		$\bar{z}$ 0	z 1
$x \ y$	$\bar{x}\bar{y}$ 00	0 000	1 001
	$\bar{x}y$ 01	0 010	1 011
	xy 11	0 110	1 111
	$x\bar{y}$ 10	0 100	1 101

Simplifying Boolean Expressions

- Cell grouping of 4
- Result is 1 when z is 1
 - x and y are irrelevant
- Result:

$$x\bar{y}z + xyz + \bar{x}yz + \bar{x}\bar{y}z = \mathbf{z}$$

$\begin{matrix} & \mathbf{z} \\ \mathbf{x} \ \mathbf{y} & \swarrow \end{matrix}$		$\bar{z}$	z
		0	1
$\bar{x}\bar{y}$	00	0 000	1 001
$\bar{x}y$	01	0 010	1 011
xy	11	0 110	1 111
$x\bar{y}$	10	0 100	1 101

Simplifying Boolean Expressions

- Another (new) SOP Expression:

$$x\bar{y}\bar{z} + x\bar{y}z + \bar{x}\bar{y}\bar{z} + \bar{x}\bar{y}z$$

		z	
		$\bar{z}$ 0	z 1
x y	$\bar{x}\bar{y}$ 00	1 000	1 001
	$\bar{x}y$ 01	0 010	0 011
	xy 11	0 110	0 111
	$x\bar{y}$ 10	1 100	1 101

Simplifying Boolean Expressions

- Cell groups can wrap around the map
- Result is 1 when y is 0
 - x and z are irrelevant

- Result:

$$x\bar{y}\bar{z} + x\bar{y}z + \bar{x}\bar{y}\bar{z} + \bar{x}\bar{y}z = \bar{y}$$

$x \ y \backslash z$		$\bar{z}$	z
		0	1
$\bar{x}\bar{y}$	00	1 000	1 001
$\bar{x}y$	01	0 010	0 011
xy	11	0 110	0 111
$x\bar{y}$	10	1 100	1 101

Simplifying Boolean Expressions

- The usefulness of K-Maps for simplifying Boolean expressions should be apparent
- However, K-Maps will not explicitly show:
 - NAND Operations
 - NOR Operations
 - XOR Operations
 - XNOR Operations

Simplifying Boolean Expressions

- No groupings in this K-Map

- Result:

$$\bar{x}\bar{y} + xy$$

- Must recognize as XNOR

$$x \odot y$$

		y	
		$\bar{y}$	y
x	$\bar{x}$ 0	1 00	0 01
	x 1	0 10	1 11

Simplifying Boolean Expressions

- Another ability of K-Maps are that it can ignore certain outputs.
- These outputs are called **don't cares**
- If there are certain inputs that we don't care about, we can still use those inputs in the simplification process

Simplifying Boolean Expressions

- Starting with a truth table...
- F represents the output of this function
- X represents outputs we don't care about
 - We don't what the output is for $x\bar{y}\bar{z}$ or $xy\bar{z}$

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	X
1	0	1	0
1	1	0	X
1	1	1	1

Simplifying Boolean Expressions

- Building the K-Map

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	X
1	0	1	0
1	1	0	X
1	1	1	1

		z	
		$\bar{z}$ 0	z 1
$x \ y$	$\bar{x}\bar{y}$ 00	0 000	0 001
	$\bar{x}y$ 01	1 010	1 011
xy 11	xy 11	X 110	1 111
	$x\bar{y}$ 10	X 100	0 101

Simplifying Boolean Expressions

- Normally, we'd get the result:

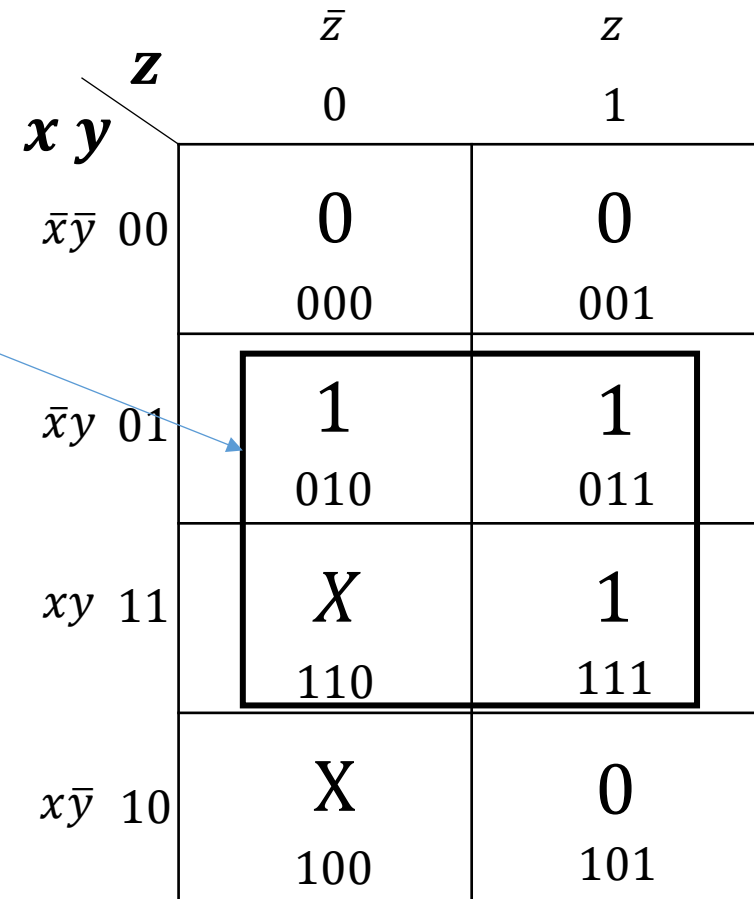
$$\bar{x}y + yz$$

- But, we can use the don't cares as a 1 if it helps simplify further

		z	
		$\bar{z}$ 0	z 1
x y	$\bar{x}\bar{y}$ 00	0 000	0 001
	$\bar{x}y$ 01	1 010	1 011
	xy 11	X 110	1 111
	$x\bar{y}$ 10	X 100	0 101

Simplifying Boolean Expressions

- Now, our result is: y



		z	
		$\bar{z}$ 0	z 1
$x \ y$	$\bar{x}\bar{y}$ 00	0 000	0 001
	$\bar{x}y$ 01	1 010	1 011
	xy 11	X 110	1 111
	$x\bar{y}$ 10	X 100	0 101

Simplifying Boolean Expressions

- Checking with the truth table...
- Again, this function F doesn't care about the output for the inputs of $x\bar{y}\bar{z}$ or $xy\bar{z}$

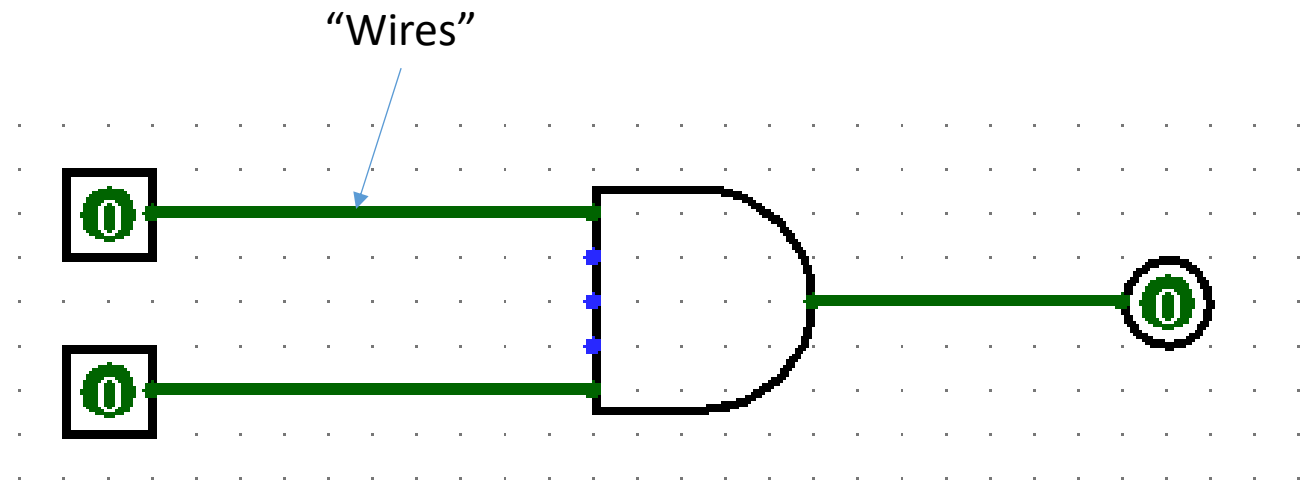
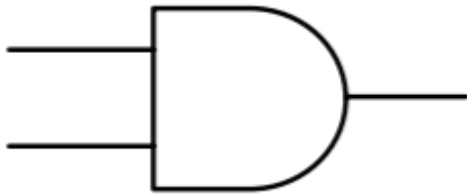
x	y	z	F	$\bar{x}y$	yz	$\bar{x}y + yz$
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	1	1	0	1
0	1	1	1	1	1	1
1	0	0	X	0	0	0
1	0	1	0	0	0	0
1	1	0	X	0	0	0
1	1	1	1	0	1	1

Logic Circuits

- The Boolean expressions we have been working with are the basis of constructing logic circuits.
- A **logic circuit** is a diagram of a Boolean expression.
- Logic circuits are built using **logic gates** that perform the different logical operations

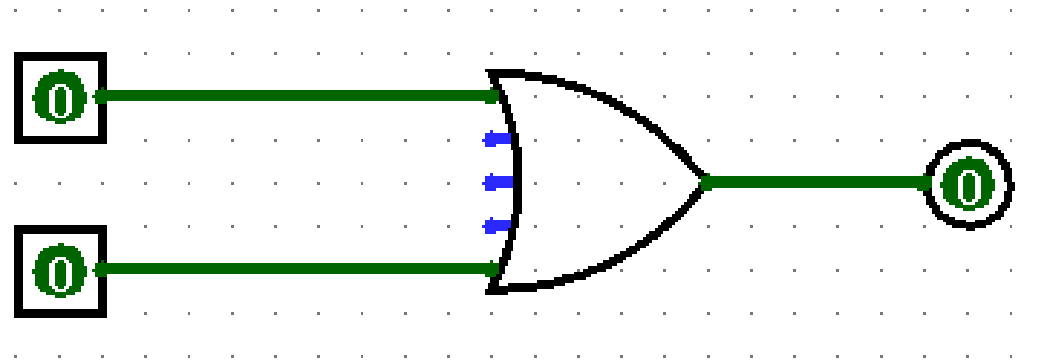
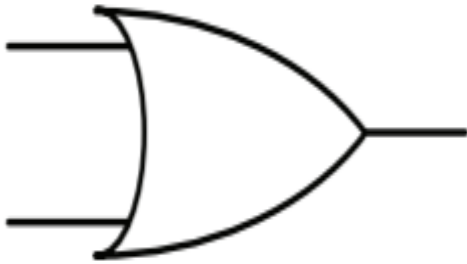
Logic Circuits

- The AND Gate



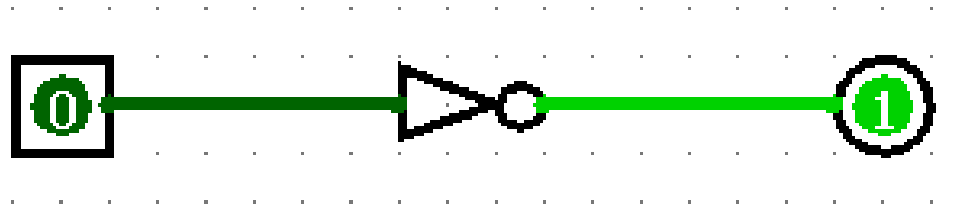
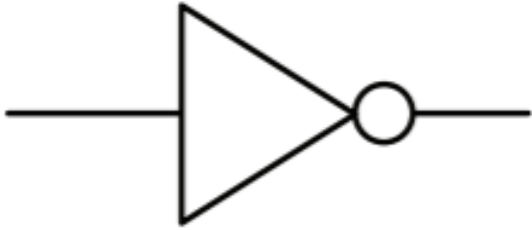
Logic Circuits

- The OR Gate



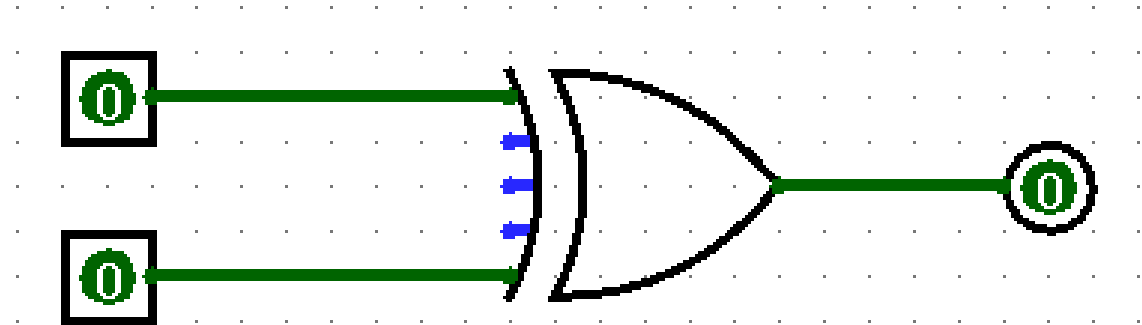
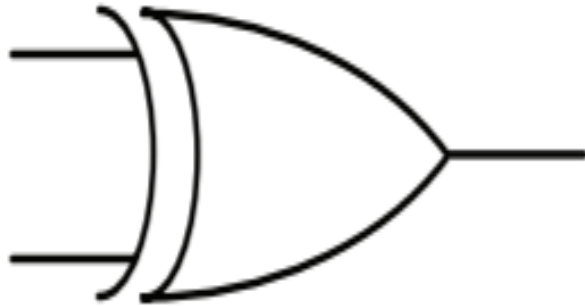
Logic Circuits

- The NOT Gate



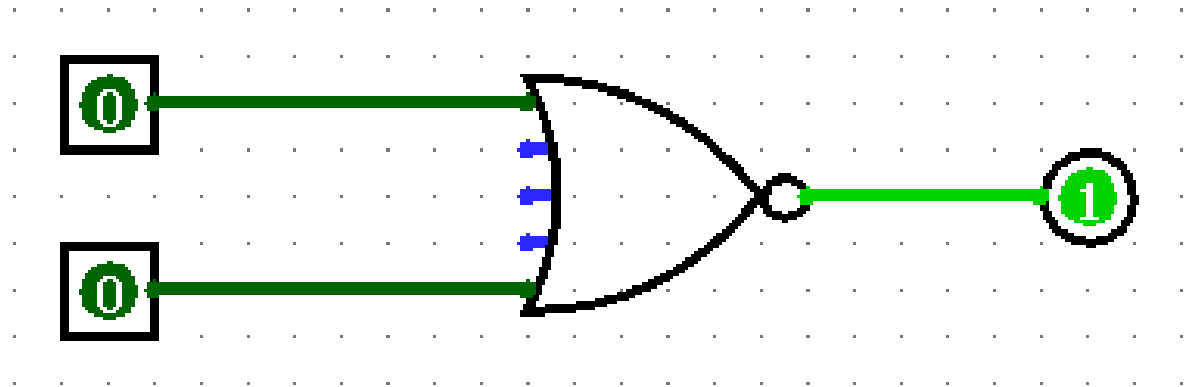
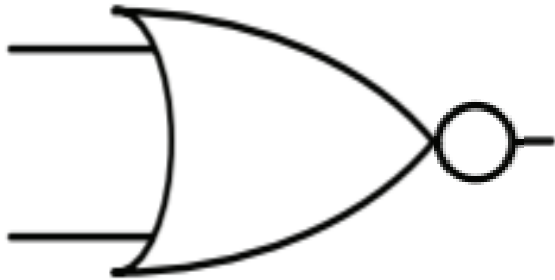
Logic Circuits

- The XOR Gate



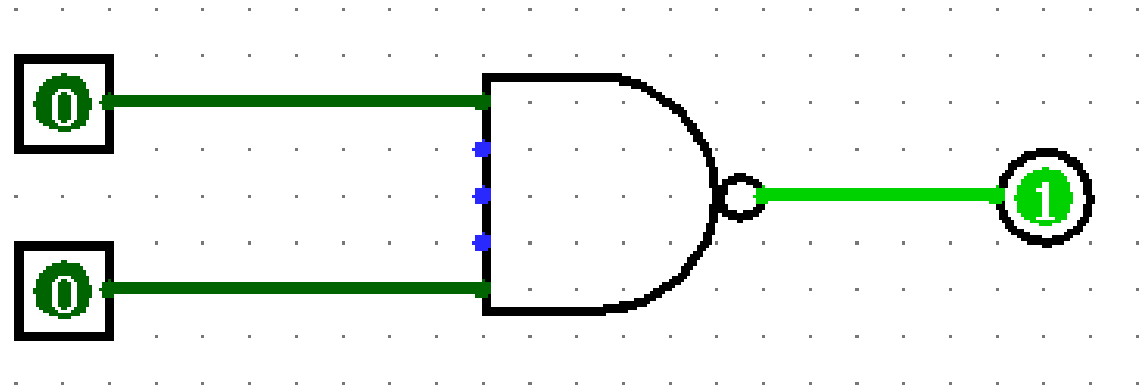
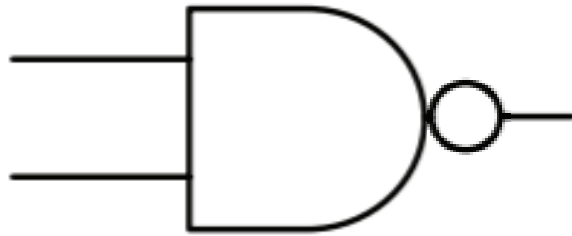
Logic Circuits

- The NOR Gate



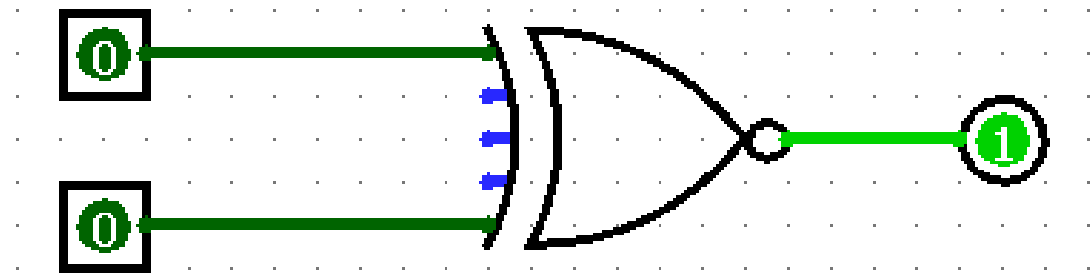
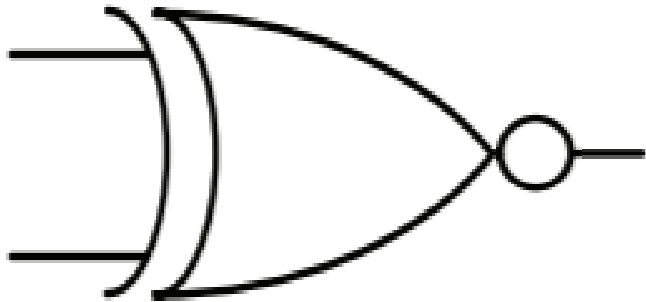
Logic Circuits

- The NAND Gate



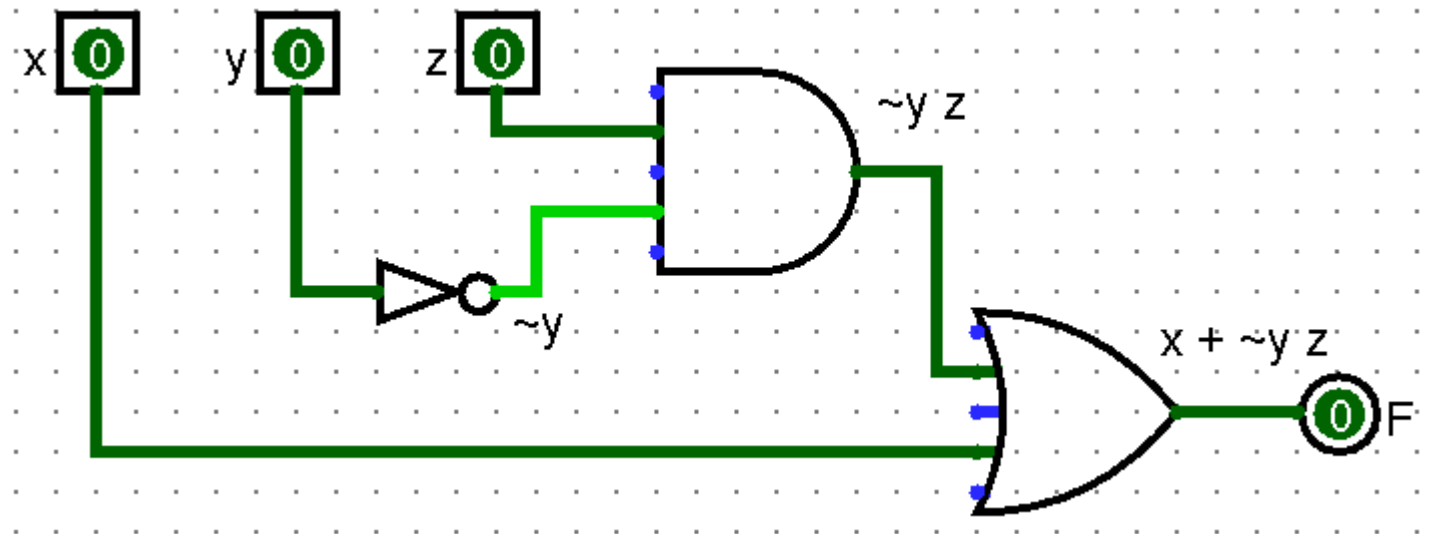
Logic Circuits

- The XNOR Gate



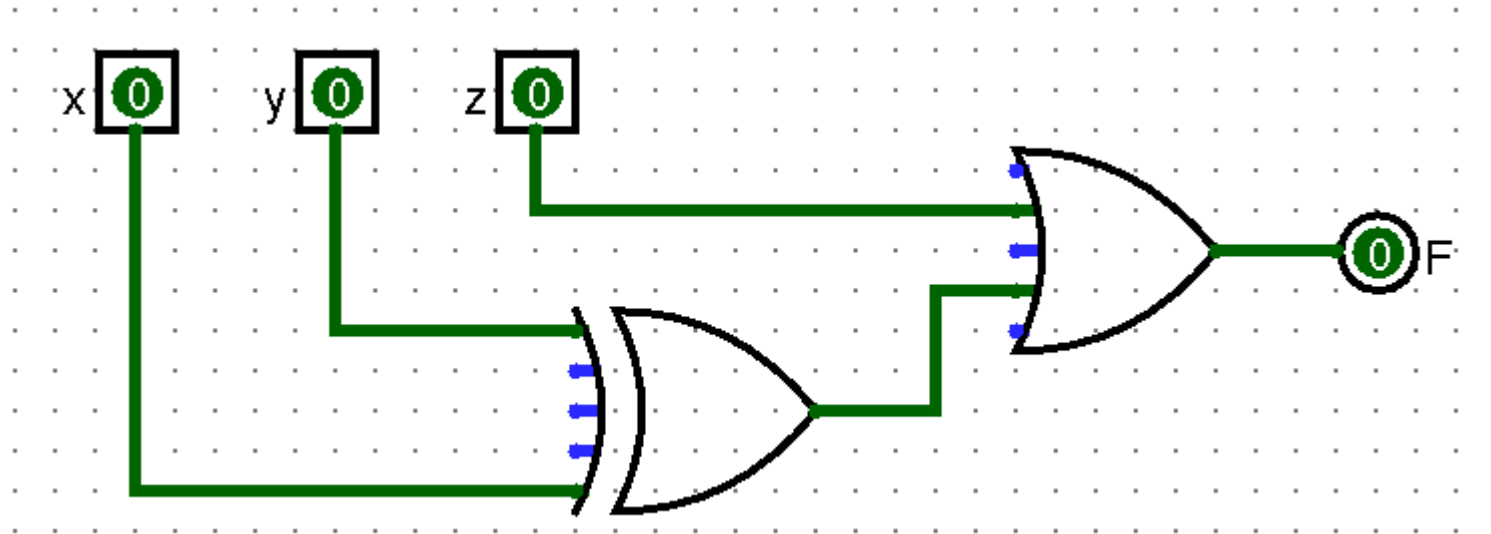
Logic Circuits

- A logic circuit for the Boolean expression $x + \bar{y}z$
 - Be sure to follow order of operations



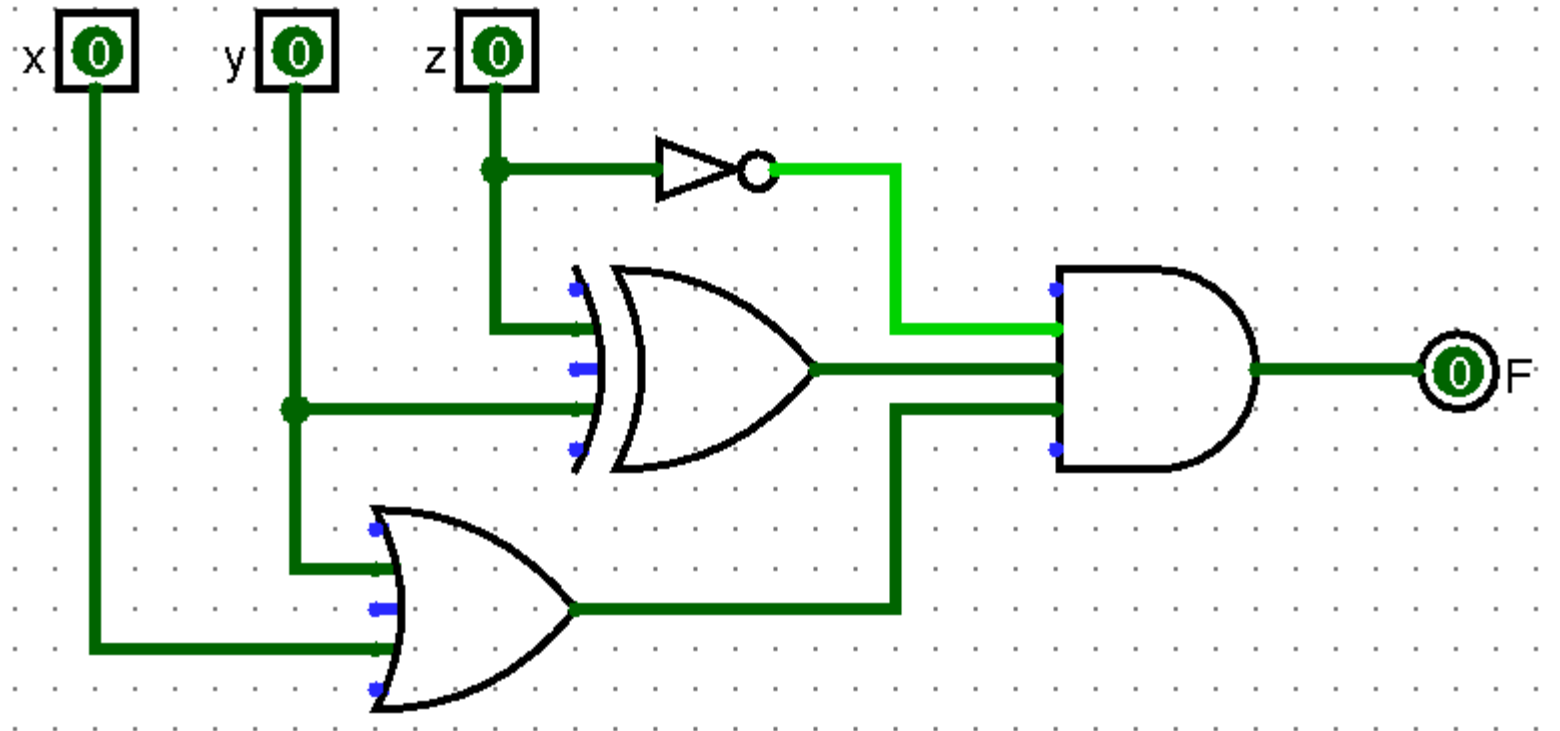
Logic Circuits

- A logic circuit for the Boolean expression $x \oplus y + z$



Logic Circuits

- A logic circuit for the Boolean expression $(x + y)(y \oplus z)\bar{z}$

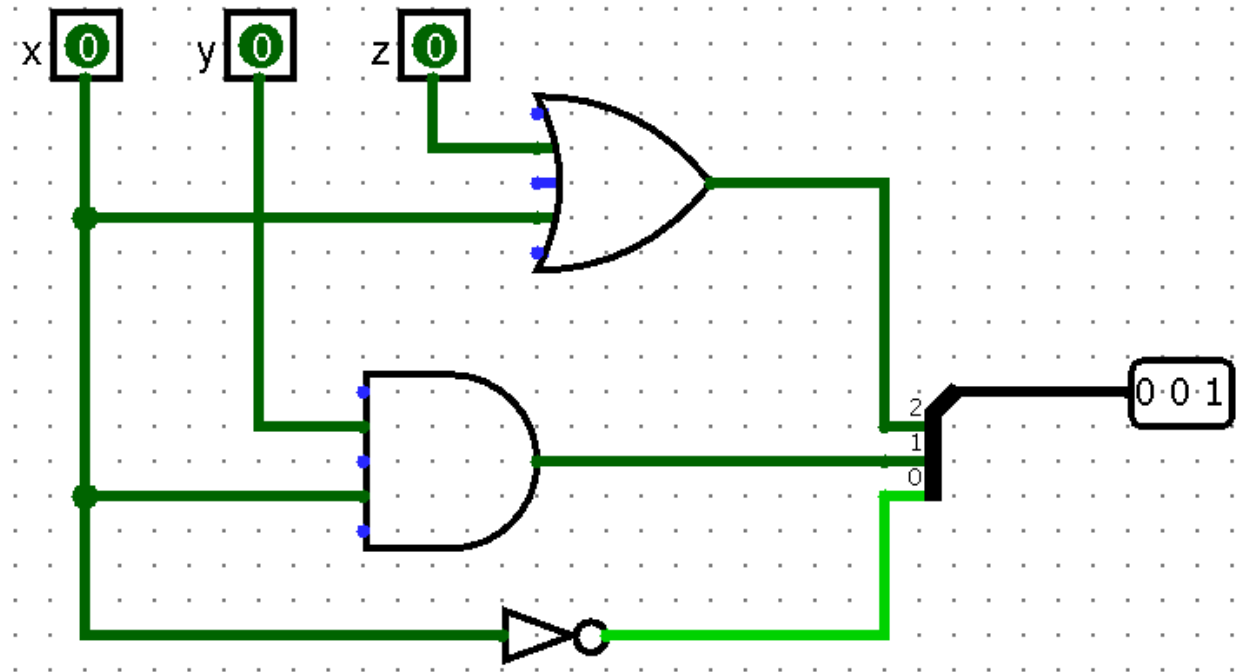


Buses

- A **bus** is several parallel wires going from one component to another.
- The bus's **width** is the number of parallel wires
- In a logic circuit, many parallel wires (like, say in a 32-bit bus) may make the diagram large and confusing

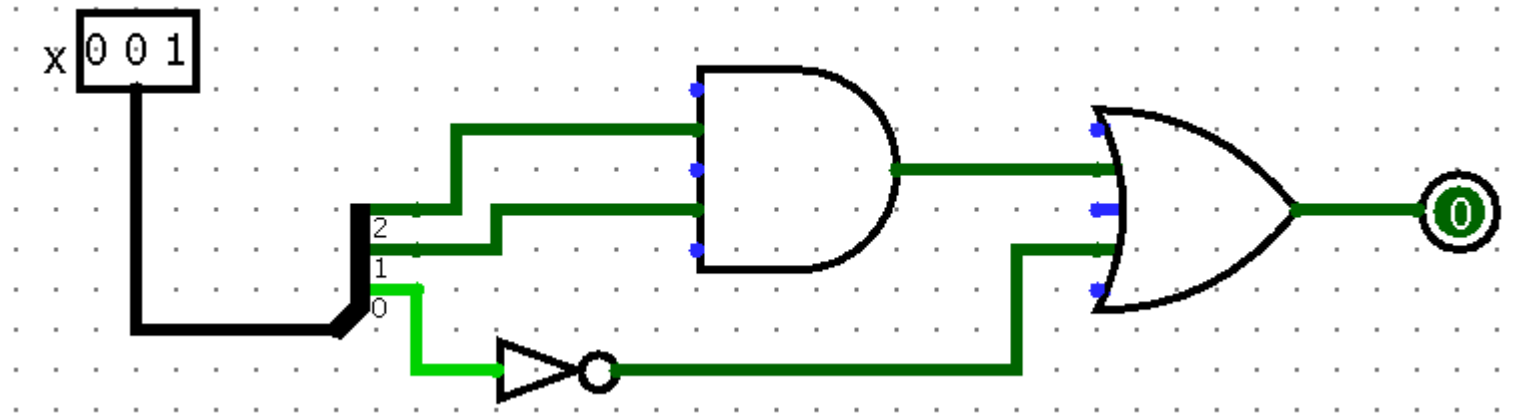
Buses

- This circuit illustrates bundling three wires into one 3-bit bus



Buses

- This circuit illustrates splitting the wires from a 3-bit bus

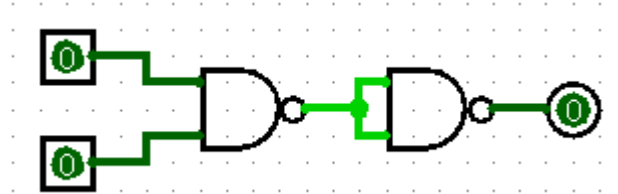


NAND-only/NOR-only Logic Circuits

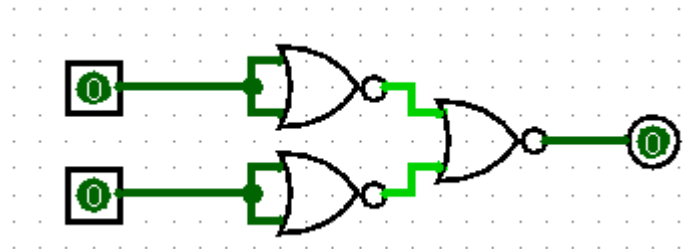
- All the previously shown gates can be implemented using only NAND gates or implemented using only NOR gates
- NAND and NOR gates require fewer transistors than other gates, thus only using these gates will save room

Logic Circuits

- NAND-only AND gate:

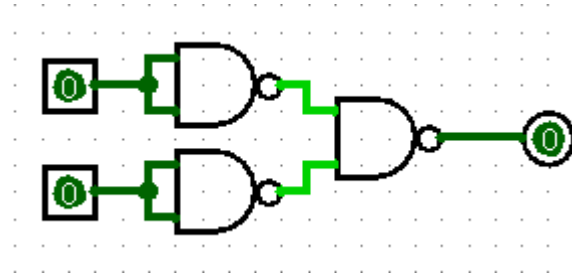


- NOR-only AND gate:

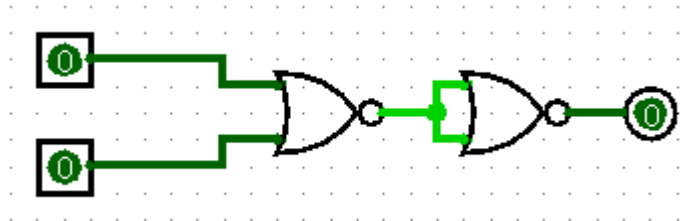


Logic Circuits

- NAND-only OR gate:



- NOR-only OR gate:



Logic Circuits

- NAND-only NOT gate:

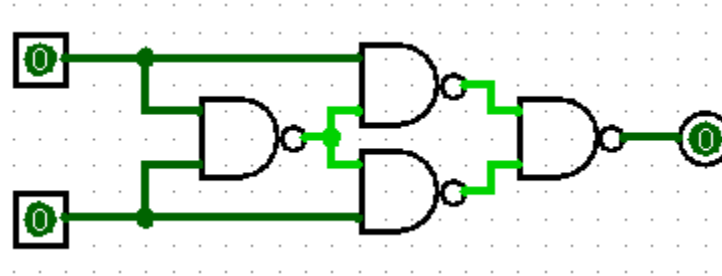


- NOR-only NOT gate:

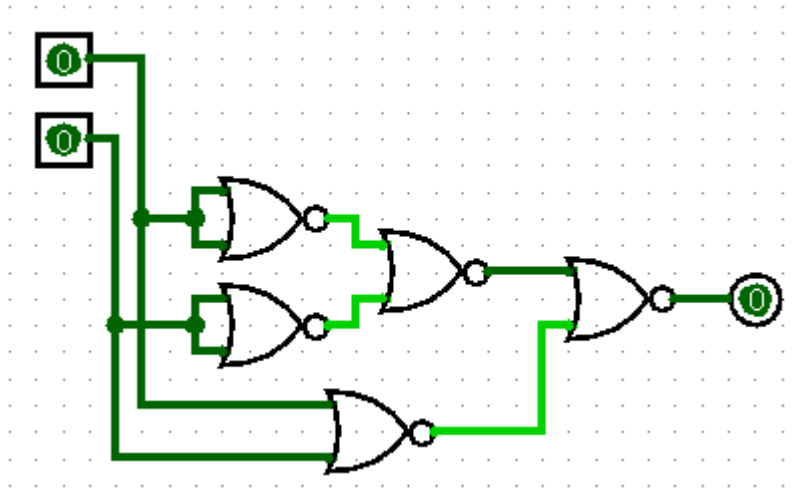


Logic Circuits

- NAND-only XOR gate:

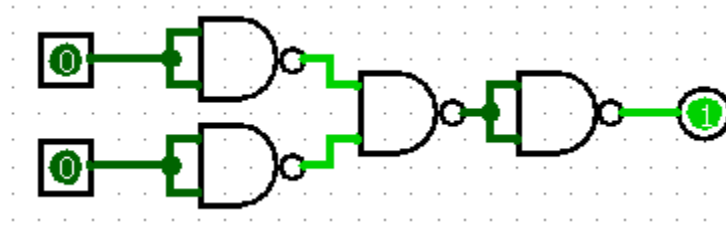


- NOR-only XOR gate:

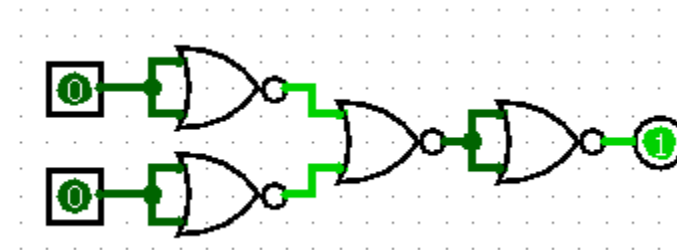


Logic Circuits

- NAND-only NOR gate:

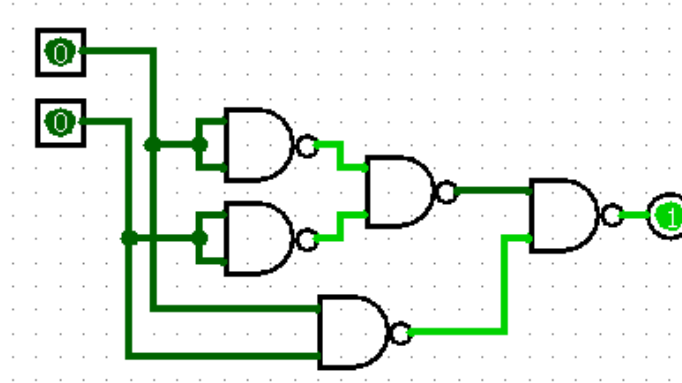


- NOR-only NAND gate:



Logic Circuits

- NAND-only XNOR gate:



- NOR-only XNOR gate:

